

ROT: Designing a 3,000-Line Tree-Walking Interpreter

Omkar Patel

Claude Opus 4.7 (1M context)

May 2026 — ROT v2.25.12

Abstract

ROT is a small, hand-rolled programming language — a learning project and portfolio piece — whose current release (v2.25.12) ships a tree-walking interpreter, a recursive-descent parser, lexically-scoped closures with explicit shadowing, classes with bound methods, error handling with `finally`, slicing, f-strings with format specs, a module system, an interactive REPL, and `rustc`-style errors. The implementation is about 3,000 lines of Python with 628 passing tests on Python 3.9 through 3.12. This paper is a design retrospective. It walks through ROT’s pipeline, dwells on the two design choices that shaped most of the code (chain-walking assignment and source-located errors), and pulls out the lessons that turned out to be worth more than the language itself.

1 Introduction

ROT started as a regex transpiler in v1.0.0: hand a string to a giant pattern table, swap a few tokens, and `exec()` the result as Python. That was instructive for about a week. It bottomed out the moment the language needed anything Python’s syntax couldn’t fake — different scoping rules, custom error messages, a real call model. So the project pivoted. The transpiler was replaced by a hand-rolled lexer in v1.6.0, an AST in v1.7.0–v1.8.0, an emitter in v1.9.0, and finally a true tree-walking interpreter in v2.0.0 that dropped `exec()` entirely. By v2.25, ROT had grown a complete control-flow surface (loops, `break/continue`, exceptions with `finally`), user classes, an explicit-shadow `let` keyword, slicing, f-string format specs, a small module system, and a REPL with persistent history. The language’s source weight is roughly 3,000 lines; the test suite is roughly 6,000.

The point of ROT is not to be production-grade — it’s a tree-walker, so it’s inherently slower than even an interpreted bytecode VM. The point is to own the entire pipeline end-to-end and reason about each design decision under the assumption that you might have to live with it. Hosting a language inside Python is convenient until the host starts leaking: a stray `getattr` call gives the user `__class__` on every string, a `break` signal walks past a function boundary into the caller’s loop, a Python `ZeroDivisionError` surfaces with a Python traceback through your nice `try/catch` block. Each of these is a small bug. Together they answer a real question: what does it mean to actually *contain* a language?

The rest of this paper takes the architecture in order — surface syntax, the four-stage pipeline, the environment model, control-flow signals, error rendering, and the information-leak hardening pass that closed the host gap — and ends with a few lessons that generalize past the language itself.

2 Surface Syntax

ROT’s surface is a deliberate hybrid. The braces and the type-of-thing flavor are from C++; the dynamism is from Python; the quirks are mine.

- `funct` for function definition, not `def`. The name is short and unambiguous.

- `cout` and `coutln` for printing. `coutln` adds a newline; `cout` does not. Homage to `std::cout`.
- `|` as the parameter and argument separator, not `,`. Pure stylistic choice from the v1.0 design; kept because it forces every reader to notice they're not reading Python.
- `this`, not `self`. C++ camp on this one.
- `//` for comments, C-style.
- Curly-brace blocks. No significant indentation.
- Lowercase identifiers only (`[a-z_][a-z_0-9]*`). Keywords are lowercase; identifiers match.
- Built-in literals: `true`, `false`, `null`.

A side-by-side helps. Here's a small program that filters and sums; on the left in ROT, on the right in Python:

```

func sum_positive(xs) {
    total = 0
    for x in xs {
        if (x > 0) {
            total += x
        }
    }
    return total
}

coutln(sum_positive([1 | -2 | 3 | -4]))

```

```

def sum_positive(xs):
    total = 0
    for x in xs:
        if x > 0:
            total += x
    return total

print(sum_positive([1, -2, 3, -4]))

```

The surface earns its keep by being immediately readable to anyone who has used C++ or Python — but readable in a way that signals it isn't either of them. That matters for a language whose semantics differ from both (see Sections 4 and 5).

3 The Pipeline

ROT runs source through four stages: lex, parse, evaluate, execute. There's no intermediate Python; `exec()` has been gone since v2.0.0. Figure 1 sketches the data flow for a one-line program.

The lexer (`rot/lexer.py`, ~320 lines) is hand-rolled and character-by-character. It dispatches on the next character — digit, letter, quote, slash, etc. — and emits a `Token` carrying (`lexeme`, `kind`, `line`, `col`). Keywords are looked up in a small dict after a lowercase-letter run is consumed; everything else becomes `IDENT`. The lexer raises `LexerError(line, col)` on illegal characters instead of silently dropping them. It also threads source position through every token; the parser and interpreter rely on this for error rendering (Section 6).

The parser (`rot/syntax.py`, ~870 lines) is recursive-descent for statements and Pratt-style for expressions. There's one method per grammar rule (`_parse_func_def`, `_parse_if_stmt`, `_parse_atom`, and so on) and one precedence table for infix operators. Every AST node carries the `line/col` of its first token, which is what makes `rustc`-style error rendering possible later.

The interpreter (`rot/interpreter.py`, ~990 lines) walks the AST directly. There are two dispatchers — one over statement node types, one over expression node types. Functions are `RotFunction` objects that hold their declaration and a snapshot of the closure environment. Classes are `RotClass`; instances are `RotInstance`; methods bound to an instance are `BoundMethod`. The environment is a single mutable type, `Environment`, with one quirk in its `set` method — which is the topic of Section 4.

A standalone AST-to-Python emitter (`rot/emitter.py`) lived from v1.9.0 to v2.22. It compiled to Python and used `exec()`. After v2.0.0 dropped `exec()` from the main path, the emit-

```

"coutln(1 + 2)"                                <-- source string

v Lexer (rot/lexer.py)

[Token(PRINTLN, "coutln"),
Token(L_PAREN, "("),
Token(NUMBER, "1"),
Token(ADDITION, "+"),
Token(NUMBER, "2"),
Token(R_PAREN, ")")]

v Parser (rot/syntax.py)

Program(body=[
  ExprStmt(Call(
    callee=Identifier("coutln"),
    args=[BinaryOp("+", NumberLit(1), NumberLit(2))]
  ))
])

v Interpreter (rot/interpreter.py)

- _execute_statement(ExprStmt)
- _evaluate(Call)
  - resolve "coutln" -> Python callable in builtins env
  - _evaluate each arg
    - _evaluate(BinaryOp "+", 1, 2) -> 3
  - invoke callable(3)

v stdout

"3\n"

```

Figure 1: End-to-end data flow for `coutln(1 + 2)`.

ter became dead weight, and a v2.13.0 audit cataloged about 40 drift items in it: missing `nonlocal` declarations broke the v2.10.0 closure-mutation feature; `null/true/false` were emitted as Python's `None/True/False` and printed wrong; `assert(cond, msg)` was emitted as Python's `assert (False, 'msg')`, a two-tuple that is always truthy. The emitter was removed wholesale in v2.23.0. A tree-walking interpreter is the right *reference* implementation: it directly encodes the language's semantics, which means you can read the language by reading the interpreter. Any future bytecode VM (Section 10) will be tested against its outputs.

4 Environment Design

The interesting design problem in any small interpreter is name binding. ROT's solution is two methods on a single `Environment` type — `set` and `set_local` — plus a top `frozen` layer for builtins and an explicit `let` keyword for opt-out shadowing. Each piece exists because of a specific bug or footgun that came before it.

4.1 Chain-walking `set`

`Environment.set(name, value)` walks the parent chain. If `name` is already bound anywhere up the chain, the existing binding is mutated; otherwise, a new binding lands in the current scope. The whole point of this — introduced in v2.10.0 — is that closures can mutate the enclosing scope by

default:

```
funcnt make_counter() {
    count = 0
    funcnt inc() {
        count += 1    // walks up, finds outer 'count', mutates it
        return count
    }
    return inc
}

c = make_counter()
coutln(c())    // 1
coutln(c())    // 2
```

Without chain-walking, this would silently create a local `count` inside `inc` and Python would crash with `UnboundLocalError` on `count += 1` — which is exactly the well-known Python footgun that `nonlocal` was invented to patch. ROT’s design says: closure-mutation is the default; the trade-off is that you cannot shadow an outer variable just by reusing its name.

4.2 set_local at declaration sites

Chain-walking is the right default for assignment. It is the wrong default for declarations. Imagine:

```
funcnt outer() {
    funcnt work() { return 1 }
    funcnt inner() {
        funcnt work() { return 2 }    // outer.work suddenly returns 2 also?
        return work()
    }
}
```

Treating `funcnt work` as an assignment makes the nested declaration chain-walk into the outer scope and silently rebind `outer.work` too. That’s not a closure — that’s a name-collision bug masquerading as one. The v2.13.0 audit catalogued this as findings I15 and I16 (nested function and class declarations clobbering outer same-named ones).

The fix in v2.16.x was to use a different method, `set_local`, at every declaration site:

- `funcnt` declarations.
- `class` declarations.
- Method parameters and `this` in `BoundMethod.call`.
- Function parameters in `RotFunction.call`.
- `for x in` `iter` loop variables.
- The `catch (e)` binding (which otherwise would clobber a global `e`, including the math constant).

`set_local` binds in *this* scope, full stop — no chain walk. The principle: *declarations are local; only user assignment walks the chain*.

4.3 The frozen builtins layer

Even with `set_local` at declaration sites, plain `pi = 3.0` would still chain-walk up the `env` tree, hit the builtin `pi`, and silently overwrite it. The fix in v2.16.5 was to put builtins in their own `Environment` with `frozen=True`, and root the user’s globals as a child of it. Writes that walk the chain into the frozen layer are rejected:

```

pi = 3.0
// error: cannot reassign builtin 'pi'

let pi = 3.0    // ok -- 'let' declares a new local, no chain walk
coutln(pi)      // 3.0

```

This is the only place in the interpreter where a write can fail for a reason other than "you tried to assign to a non-name." It's also why the builtins live in a separate **Environment** rather than being injected into the user's globals: the **frozen** bit is per-environment, not per-key.

4.4 The **let** keyword

What if the user actually *does* want to shadow an outer name — not by accident, but on purpose? Adding **let** in v2.16.6 gave them an explicit way to opt out of chain-walking:

```

func shadow_demo() {
  x = 1
  func inner() {
    let x = 99    // explicit fresh local; no chain walk
    coutln(x)    // 99
  }
  inner()
  coutln(x)      // 1 -- outer x is untouched
}

```

let name = expr runs `env.set_local(name, value)` regardless of what's in the parent chain. It's the explicit form of "I know there's a same-named variable above me; I want my own." Bare `=` keeps the v2.10.0 closure-mutation semantics. The pair — implicit chain-walk for assignment, explicit **let** for shadowing — separates the two intents.

4.5 The honest trade-off

Chain-walking assignment is not strictly better than fresh-local assignment. It buys you closure-mutation without ceremony, but it also buys you action-at-a-distance: if a function deep inside a call chain assigns to a name that happens to exist in an enclosing scope, it mutates that enclosing scope. In a long call chain across modules, this can be hard to debug. The v2.16.6 design point is that ROT prefers the trade because the most common case — counters, accumulators, building up state in a closure — is more ergonomic, and the rare case (deliberate shadow) is covered by an explicit keyword. Python made the opposite choice (**nonlocal** as opt-in mutation). Both are defensible; ROT's choice is the one we made.

5 Control-Flow Signals

break, **continue**, **return**, and **throw** all need a way to unwind the AST walker non-locally. The choice ROT makes is the simplest one: each of them raises a Python **BaseException** subclass that the relevant statement handler catches.

```

class _ReturnSignal(BaseException):
  def __init__(self, value): self.value = value

class _BreakSignal(BaseException): pass
class _ContinueSignal(BaseException): pass

class _ThrowSignal(BaseException):
  def __init__(self, value, line=0, col=0):

```

```

self.value = value
self.line = line
self.col = col

```

They subclass `BaseException` (not `Exception`) deliberately: a user `try/catch` translates to a Python `try/except Exception`, so the control-flow signals slip past it as they should. `return` bubbles up to whichever `RotFunction.call` or `BoundMethod.call` is on the Python stack, where `except _ReturnSignal` catches it and returns the carried value. `break` and `continue` bubble up to the nearest `WhileStmt` or `ForStmt` handler.

5.1 The lexical-scope bug

A naïve implementation has a quiet bug. Picture the interpreter holding a single `_loop_depth` counter (incremented on `while/for`, decremented on exit), and a function body that does `break`:

```

funct quit() { break }

for i in range(5) {
    coutln(i)
    quit()
    coutln("after quit")
}

```

What you want: a clear error — ‘`break`’ outside of a loop — because `quit`’s body has no enclosing loop. What you get without the fix: `break` raises `_BreakSignal`, which propagates straight up through `quit`’s `RotFunction.call` (no `except` for it), past `_evaluate_call`, up to the `for` loop body — which catches `_BreakSignal` and exits. The output is just 0, then nothing. `break` crossed a function boundary into the caller’s loop. The v2.13.0 audit logged it as finding I1.

The fix (v2.15.1) sits in the two function-call sites. They save and zero out `_loop_depth` on entry, restore it in a `finally` block, and explicitly catch any `break` or `continue` signal that escapes the body, re-raising it as a clean error:

```

prior_loop_depth = interpreter._loop_depth
interpreter._loop_depth = 0
# ... execute body ...
try:
    interpreter._execute_block(self.decl.body)
except _ReturnSignal as signal:
    return signal.value
except _BreakSignal:
    raise InterpreterError("'break' outside of a loop")
except _ContinueSignal:
    raise InterpreterError("'continue' outside of a loop")
finally:
    interpreter._loop_depth = prior_loop_depth

```

The same idea applies at the top of `Interpreter.execute`: an uncaught `_ThrowSignal` would otherwise propagate as a raw `BaseException` with a Python traceback. The outer `try/except _ThrowSignal` catches it and re-raises an `InterpreterError("uncaught throw: ...", line, col)`. The principle is the same in both places: *a control-flow signal that’s not lexically valid here must surface as a clean user-facing error, never as a Python traceback.*

6 Error Rendering

Errors in ROT look like `rustc`'s. They name the file, point at the line, draw a caret under the column, and offer a hint when the user typed a Python-ism:

```
error: name 'print' is not defined (did you mean 'cout'?)
--> hello.rot:1:1
    |
1 | print("hi")
    | ^
```

The before-and-after is starker than it sounds. The v2.13.0-era error for the same input was three Python frames of traceback ending in `NameError: name 'print' is not defined` — no source line, no caret, no hint. Getting from there to here took two patches that turned out to matter past ROT itself.

6.1 The `_locate` dispatcher pattern

Every AST node carries `line/col`, and most `InterpreterError` raise sites carry the position of the relevant node. But several errors are raised deep inside helper methods — `Environment.get` for an undefined name, builtin wrappers that receive arguments without nodes, Python interop errors that surface inside `getattr`. There are about 30 such raise sites, scattered across the file. Going site by site and threading a position into each one is a lot of edits and a lot of new bugs.

The pattern in v2.22.3 was instead: wrap `_evaluate` and `_execute_statement` in thin dispatchers that catch any position-less `InterpreterError` and patch it with the current AST node's position. One wrapper, one helper:

```
def _evaluate(self, expr):
    try:
        return self._evaluate_impl(expr)
    except InterpreterError as e:
        raise self._locate(e, expr)

def _locate(self, err, node):
    if getattr(err, "line", 0):
        return err # inner site knew better -- keep its position
    line = getattr(node, "line", 0) if node is not None else 0
    col = getattr(node, "col", 0) if node is not None else 0
    if not line:
        return err
    return type(err)(err.message, line=line, col=col)
```

The hook is one function. The 30 raise sites stay as they were. Errors raised from `Environment.get`, from builtin call failures, from `getattr` interop — all get located on whatever AST node was being evaluated at the time. Inner errors with their own position are left alone, because the inner site knew its position better than the outer wrapper does.

6.2 The renderer

The actual block layout is in `RotError.format(source, filename)` in `rot/errors.py`. Given an error with a `(line, col)`, the source string, and the file name, it produces the header line, the source line numbered to match, and a caret under the offending column. The line-number gutter is padded to the digit width of the line number so multi-line files align cleanly. When there's no location (`line == 0`), it falls back to a bare `error: <message>`. The CLI and REPL both call

`err.format(...)` instead of `f"rot error: {err}";` `str(err)` still produces the old "line N:C:msg" format, so the existing test suite stays green.

6.3 Python-ism hints

A common user mistake on ROT is typing Python: `print(...)`, `None`, `True`, `def`, `self`. The v2.22.6 patch added a small `_PYTHON_HINTS` table to `Environment.get`: on a name miss, if the missing name is a recognizable Python identifier, the error message appends (did you mean 'X'?). Concretely:

```
name 'print' is not defined (did you mean 'cout'?)
name 'None' is not defined (did you mean 'null'?)
name 'def' is not defined (did you mean 'funct'?)
name 'self' is not defined (did you mean 'this'?)
```

Plain unknown names don't get the suffix — the table is a closed set. The honest principle: don't pretend to know what the user meant; suggest only when the suggestion is confidently correct.

7 Information-Leak Hardening

Hosting a language inside Python is convenient, and it's also a security surface. Every time the interpreter falls through to a Python operation — `getattr`, a builtin's method, an attribute lookup — there's a question of what the user sees through that channel. The v2.18.x sweep closed every leak it could find.

7.1 The `__class__` leak

The original `MemberAccess` handler had a graceful fallback: if the target wasn't a `RotInstance`, just delegate to Python's `getattr`. That meant you could read Python attributes off any string, list, or dict:

```
"abc".__class__           // <class 'str'>
"abc".__class__.__bases__ // (<class 'object'>,)
[1 | 2].__init__          // <method-wrapper '__init__' of list object>
"a".encode()              // b'a' <-- a Python bytes object
```

That's a sandbox escape in spirit. Even without active hostility, it pollutes the language with concepts it never intended to expose: Python's type tree, metaclass internals, `bytes`. None of those exist in ROT. The fix in v2.18.x is three parts.

Part one: reject member names starting with `_`. Dunder and private Python conventions are out of scope for ROT, so blocking them is cheap.

```
if expr.member.startswith("_"):
    type_name = _builtin_type(target)
    raise InterpreterError(
        f"no member {expr.member!r} on {type_name}"
    )
```

Part two: give `RotClass` and `BoundMethod` their own `get_member` that exposes nothing. Without these, even after the `_`-prefix filter, names like `A.methods`, `A.name`, `A.closure`, `a.f.decl`, `a.f.instance` would still slip through Python `getattr` fallback. The dedicated `get_member` methods give a clear error for known names (cannot access method 'X' directly on class A; call it on an instance) and reject every other lookup.

Part three: reject Python `bytes` returns at the call site. `"abc".encode()` returns a `bytes` object, which has no ROT type and prints with Python's `b'...'` repr. The fix is a post-call check:

```
if isinstance(result, (bytes, bytearray, memoryview)):
    raise InterpreterError(
        f"method {name!r} returns Python {type(result).__name__}, "
        f"which is not a ROT type"
    )
```

7.2 The principle

The cumulative effect is small but real: a ROT program cannot navigate to Python's type hierarchy, cannot read a class's internals, cannot acquire a `bytes` object, and cannot read `_`-prefixed attributes on anything. It's not a hardened sandbox — a determined user could still find an edge case in a builtin — but the obvious paths are shut. The principle generalizes past ROT: *a hosted language must not leak its host*, and the cost of meeting that property is mostly attention, not code.

8 Testing Methodology

ROT has 628 tests across six files. They divide along four axes.

Per-layer unit tests (`test_lexer.py`, `test_syntax.py`, `test_interpreter.py`) drive each pipeline stage in isolation. The interpreter file is the largest by a wide margin — 3,900 lines, exercising operator semantics, scope rules, control-flow, classes, error messages, and the closure-mutation feature with regression tests for every audit finding it touches.

End-to-end golden tests (`test_end_to_end.py`) pair every `.rot` program in `examples/` with a `.expected` file and compare captured stdout. If a refactor changes observable output anywhere, this fails first.

CLI tests (`test_cli.py`) drive `python -m rot` via subprocess: `-version`, `-no-run`, `-trace`, error exit codes, the `rustc`-style block on stderr, default-to-REPL behavior with EOF, BOM-in-source handling, and permission-denied paths. Subprocess drives matter because the CLI is the surface the user actually touches.

REPL tests (`test_repl.py`) feed inputs through stdin and check stdout/stderr. They cover multi-line continuation, state preservation across inputs, lex-error recovery, the persistent-history path, and the Ctrl-C signal-handling fix from v2.19.x.

The v2.13.0 audit drove a 12-agent / 81-commit sweep that lifted the test count from 201 to 628 over a few sessions. Each agent owned one minor-version window, cluster of bugs, and matching test additions. That sequencing matters for a particular reason: most bug-fix commits in that range also include a test that pins the new behavior. Tests are the durable artifact; the fix is the perishable one.

9 Lessons Learned

A few of these are language-design lessons; most are not.

Patterns over edits. The `_locate` dispatcher is the cleanest design decision in the codebase. Thirty raise sites needed a position. The work-style answer is to thread one through each one. The pattern answer is to wrap `_evaluate` and `_execute_statement` so a single layer patches positions on the way out. One change, every site covered, no risk of forgetting one. Across the v2.14–v2.25 sweep, the patterns that fixed dozens of findings at once (the `set_local` switch, the builtins-layer frozen env, the dunder-filter at `MemberAccess`) consistently beat per-site fixes on both line count and bug count.

Pin the buggy behavior first if you can't fix it yet. The v2.24.8 patch added an explicit test for the import-cycle bug — a test that asserted the *wrong* output as the current behavior. The point wasn't to bless it: it was to make the next change visible. When v2.25.3 fixed the cycle, exactly that test failed, and the failure described what changed. The principle: *the test suite's job is to make change visible, not to certify correctness.*

Information leak is a real security property even in a hobby language. The `__class__` leak wouldn't damage a single-user toy — there's no adversary. But "what does my language expose by default" is a property of the language, not of the threat model. Once you start treating it as a real property, the rest of the design improves: `RotClass`/`BoundMethod` grow real APIs instead of being thin wrappers around Python objects; the type-introspection builtins become honest; the language as a whole stops accidentally being Python.

Closure semantics are a real design choice, not an obvious default. Section 4 walked through the chain-walking `set` versus the explicit `let` pair. The interesting part isn't which one we picked; it's that the pick is non-trivial. Python's choice (`nonlocal`) is fine; ROT's choice (chain-walk + `let`) is also fine; making the choice without realizing you were making it would have produced a language with a quiet, recurring footgun. The audit's finding I14 was that exact case: assignment inside a function silently mutated a same-named global, and there was no `let` to opt out. The fix wasn't to change the chain walk; it was to recognize the design choice and ship the keyword that made it deliberate.

Subclass `BaseException` for control flow, not `Exception`. Using `BaseException` for `_ReturnSignal` & friends means that a user-level `try/catch` (which maps to Python's `except Exception`) can't accidentally swallow them. Using plain `Exception` would have been a debug nightmare: a `catch` that silently consumed a `return` would manifest as the function "running to completion" with a `null` return. The smaller language-design lesson generalizes: when you use exceptions as a non-local jump, make sure they don't share an ancestor with the exceptions your users can catch.

Track `(line, col)` from token zero. The single highest-leverage decision in the v1.x line was adding source position to `Token` in v1.2.0. By the time we wanted `rustc`-style errors in v2.22.7, the position was already on every token and every AST node; the rendering layer was a one-day patch. If positions hadn't been there from the start, the same patch would have been a months-long refactor across every node type and every `raise` site. The principle: *instrument early, even if you don't yet know what for.*

10 Future Work

The natural next direction is the bytecode VM. Phase 4 of the `ARCHITECTURE.md` roadmap sketches it: design a small instruction set (about 30 opcodes — `LOAD_CONST`, `ADD`, `JUMP_IF_FALSE`, `CALL`, `RETURN`), compile AST to bytecode, and write a stack-based VM in a single file. The tree-walking interpreter would stay as the reference implementation; the test suite would run against both. Reference: Bob Nystrom's *Crafting Interpreters* [1], whose Part III walks this exact path for a similar small language (Lox).

Smaller open items, in roughly the order of how often I've wanted them:

- **Inheritance and `super`.** `super` is reserved with a clear "not supported" error since v2.25.7. Adding single-inheritance is a natural minor release; multiple inheritance is intentionally out of scope.
- **Multi-line strings.** Triple-quoted, Python-style. Mostly a lexer change.
- **Integer division.** Deferred in v2.25.6 because `//` is the comment marker. An alternate token (`~/`, `\\`) was evaluated and rejected. For now, `floor(a / b)` suffices.

- **Closure late-binding.** ROT inherits Python’s footgun: a loop that creates closures over the loop variable captures by reference, so all the closures see the final value. The v2.24.5 test pinned this; a deliberate fix is open.
- **Tail-call optimization.** Currently capped at Python’s recursion limit. A real TCO pass would matter for deep recursion examples.

11 Conclusion

ROT is a small language that taught me a large number of small lessons. The visible artifact is a tree-walking interpreter with the surface of a hybrid C++/Python toy. The invisible artifact is everything I now know about names binding into the right scope, signals propagating to the right handler, errors carrying the right position from the moment they’re raised, and host languages not leaking their hosts. The next phase is a bytecode VM that runs faster; the harder phase was the one that’s already done.

References

- [1] Robert Nystrom. *Crafting Interpreters*. Genever Benning, 2021.